

BLOCKCHAIN API

SaaS REST API — Data via API Key, Autentikasi JWT

Express.js • PostgreSQL • Sequelize • Vercel

Final Project 236 — [NAMA] — [NIM]

- Latar Belakang & Rumusan Masalah
- Solusi & Fitur Aplikasi
- Teknologi yang Digunakan
- Use Case & User Flow
- ERD / Rancangan Database
- Endpoint API
- Demo Aplikasi (Postman)
- Hasil Pengujian
- Deployment di Vercel
- Kesimpulan

Latar Belakang

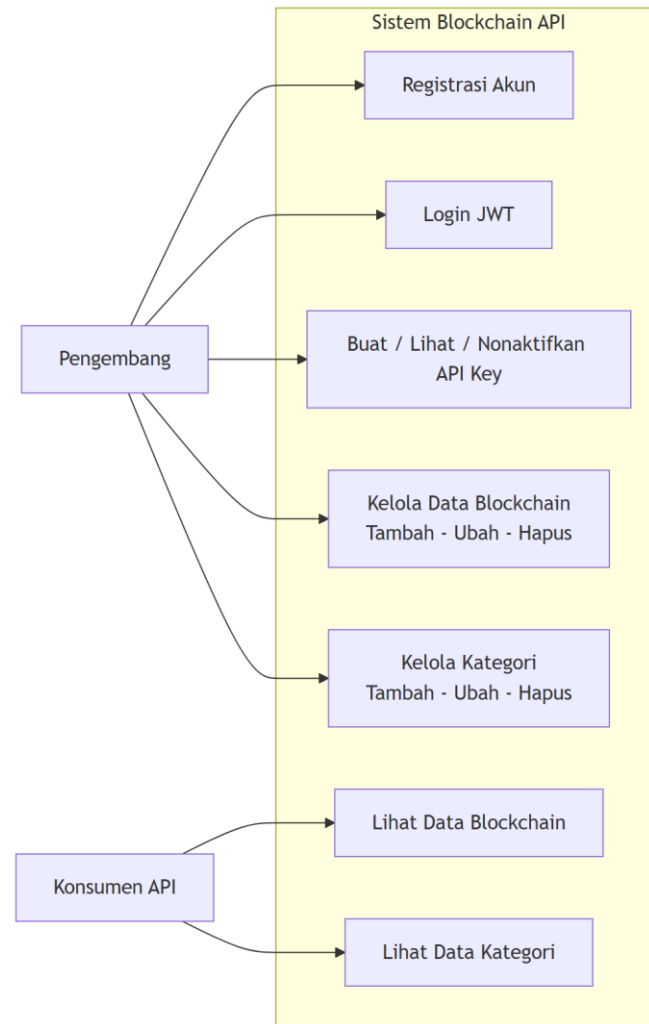
- Data blockchain semakin dibutuhkan aplikasi, namun belum tersedia dalam bentuk API siap pakai.
- Layanan SaaS seperti OpenRouter / Weather API memberikan data lewat API key — pola inilah yang diadopsi.
- Membangun REST API yang menyediakan data kepada pihak lain dengan API key + JWT.

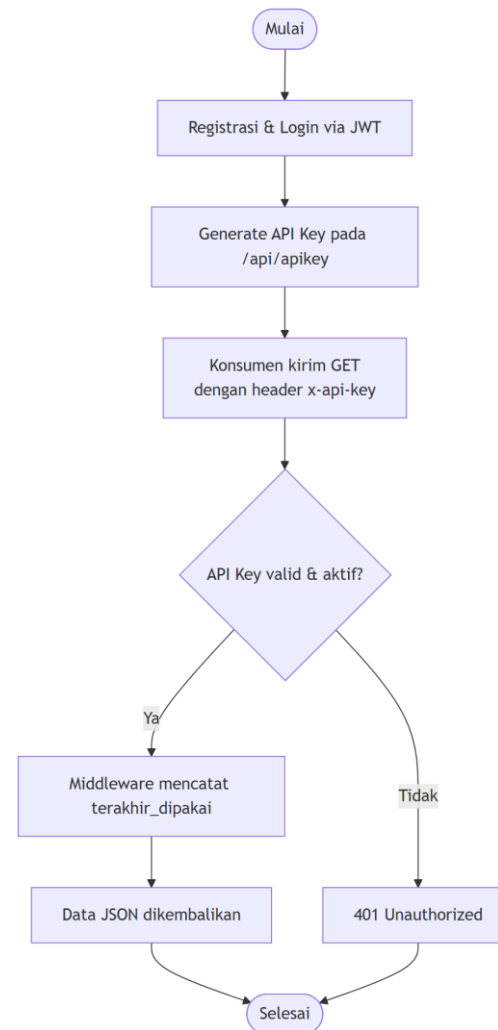
Rumusan Masalah

- Bagaimana membangun REST API SaaS berbasis API key?
- Bagaimana mengamankan akses data dengan JWT dan API key?
- Bagaimana menyediakan 50+ data kompleks dan men-deploy ke Vercel?

- **Autentikasi JWT** — login & manajemen akun pengembang (password di-hash bcrypt).
- **Sistem API Key** — konsumen cukup mengirim header x-api-key untuk mengambil data.
- **CRUD Data** — blockchain & kategori dengan relasi many-to-many.
- **60 Data Blockchain** — Bitcoin, Ethereum, Solana, Chainlink, dll. — melebihi minimal 50.
- **13 Kategori** — Layer 1, Layer 2, DeFi, NFT, Stablecoin, Meme, Oracle, dan lainnya.
- **8 Pengembang** — akun contoh untuk demo & pengujian.

Komponen	Teknologi	Fungsi
Backend	Node.js + Express.js 5	Server REST API
Database	PostgreSQL / Supabase	Penyimpanan data (5 tabel)
ORM	Sequelize 6	Model, migration, seeder
Autentikasi	JWT + bcrypt	Login & proteksi endpoint
API Key	crypto.randomBytes	Kunci akses konsumen (blk_...)
Deployment	Vercel	Hosting serverless

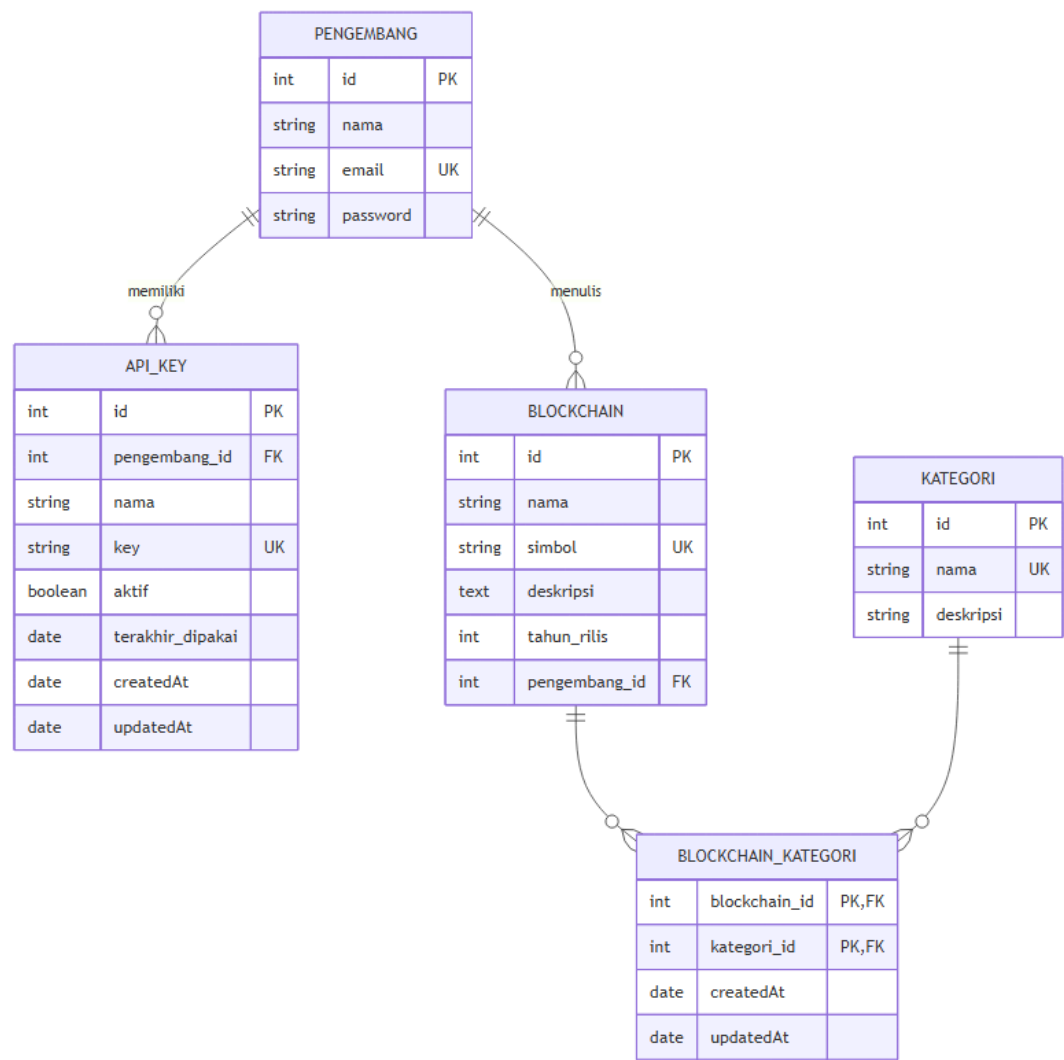




Alur konsumen:

1. Registrasi & login JWT
2. Buat API key
3. Kirim GET + x-api-key
4. Key valid → data JSON
5. Key tidak valid → 401

ENTITY RELATIONSHIP DIAGRAM (ERD)



- **pengembang** — id, nama, email, password
- **api_keys** — id, pengembang_id, nama, key, aktif, terakhir_dipakai
- **blockchain** — id, nama, simbol, deskripsi, tahun_rilis, pengembang_id
- **kategori** — id, nama, deskripsi
- **BlockchainKategori** — blockchain_id + kategori_id (tabel relasi many-to-many)

Relasi

- pengembang 1 — N blockchain (satu pengembang menulis banyak data)
- pengembang 1 — N api_keys (satu pengembang punya banyak key)
- blockchain N — N kategori (lewat tabel BlockchainKategori)

Method	Endpoint	Auth
POST	/api/register	-
POST	/api/login	-
POST	/api/apikey	JWT
GET	/api/blockchain	API Key / JWT
POST	/api/blockchain	JWT
PUT	/api/blockchain/:id	JWT
DELETE	/api/blockchain/:id	JWT
GET	/api/kategori	API Key / JWT
POST / PUT / DELETE	/api/kategori...	JWT

- **1. Register** — POST /api/register → 201
- **2. Login** — POST /api/login → token JWT (auto-simpan)
- **3. Cek akun** — GET /api/me → id, nama, email
- **4. Buat API key** — POST /api/apikey → blk_xxxxxxx
- **5. Tambah data** — POST /api/blockchain (nama, simbol, pengembang_id, kategori_ids)
- **6. Akses konsumen** — GET /api/blockchain + header x-api-key → 200, data JSON

Skenario	Hasil
Register pengembang	201 Created
Login (email & password benar)	200 + token JWT
Akses data tanpa auth	401 Unauthorized
Akses data dengan API key valid	200 + data
Akses data dengan API key salah	401
POST / PUT / DELETE dengan JWT	200 / 201

- **URL publik** — <https://236-final-project.vercel.app>
- **Cara deploy** — hubungkan repo GitHub ke Vercel → otomatis build & deploy.
- **Environment variables** — POSTGRES_URL, JWT_SECRET, JWT_EXPIRES.
- **Serverless** — tanpa kelola server, auto-redeploy tiap push.

Demo langsung di Postman dengan base URL Vercel di atas.

- **SaaS API berhasil** — data disediakan ke konsumen lewat API key (register → login JWT → buat key → akses).
- **Keamanan ganda** — JWT untuk pengelolaan, API key untuk konsumsi data.
- **60 data & 5 tabel** — relasi many-to-many — melebihi syarat minimal 50 data & 2 tabel.
- **Deploy di Vercel** — dapat diakses publik di <https://236-final-project.vercel.app>

TERIMA KASIH

Sesi Tanya Jawab

Blockchain API — Final Project 236